Reg.No : 192365001

Name : N. Sai Sreenu

CO2 – AT3

MLA0201 - FUNDAMENTALS OF MACHINE LEARNING

Debugging & Optimisation Task

Q1. Linear Regression Model for House Price Prediction

(i) Possible Causes of High Prediction Error

The high prediction error may occur because of the following reasons:

1. Multicollinearity
    - Two or more input features may be highly correlated.
    - Example: House Area and Number of Rooms.
    - This makes the regression coefficients unstable.
2. Outliers
    - Very expensive or unusually cheap houses can affect the regression line.
    - These values can increase the prediction error.
    - Important features may be missing.
    - Irrelevant features may reduce model performance.
    - Example: location, age of house, number of bedrooms, and parking availability may be important.
    - Features may have very different ranges.
    - Example: house area may be in thousands while number of bedrooms may be between 1 and 5.
    - House prices may not always have a simple linear relationship with features.
    - For example, price may increase rapidly for houses in premium locations.
    - A small dataset or incorrect/missing values can cause poor predictions.

(ii) Analyze the Dataset/Model to Detect Issues

The following methods can be used:

1. Check missing values

```
print(df.isnull().sum())
```

2. Check correlation

```
print(df.corr(numeric_only=True))
```

Highly correlated independent variables indicate possible multicollinearity.

3. Check VIF

- Variance Inflation Factor (VIF) can identify multicollinearity.
- A high VIF indicates that a feature is strongly related to other features.

4. Detect outliers

```
import matplotlib.pyplot as plt
df.boxplot()
plt.show()
```

5. Evaluate model performance

```
from sklearn.metrics import mean_squared_error, r2_score
pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, pred))
print("R2 Score:", r2_score(y_test, pred))
```

- High MSE indicates large prediction errors.
- A low R^2 score indicates poor model performance.

6. Analyze residuals

```
residuals = y_test - pred
plt.scatter(pred, residuals)
plt.xlabel("Predicted Price")
plt.ylabel("Residual")
plt.show()
```

Residuals should generally be randomly distributed around zero.

(iii) Optimization Techniques

The model can be improved using the following techniques:

1. Feature Scaling

Use StandardScaler to bring features to a similar scale.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

2. Remove Outliers

Identify unusual house prices using the IQR method and remove extreme values.

```
Q1 = df["Price"].quantile(0.25)

Q3 = df["Price"].quantile(0.75)

IQR = Q3 - Q1

df = df[(df["Price"] >= Q1 - 1.5*IQR) &

        (df["Price"] <= Q3 + 1.5*IQR)]
```

3. Remove Highly Correlated Features

If two features are highly correlated, remove one of them or use dimensionality-reduction techniques.

4. Add Relevant Features

Useful features can include:

- Location
- House area
- Number of bedrooms
- Number of bathrooms
- House age
- Parking availability
- Distance from city centre

5. Train and Test the Improved Model

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
model = LinearRegression()
model.fit(X_train_scaled, y_train)
pred = model.predict(X_test_scaled)
print("MSE:", mean_squared_error(y_test, pred))
print("R2 Score:", r2_score(y_test, pred))
```

Conclusion

The high prediction error can be caused by multicollinearity, outliers, poor feature selection, different feature scales, and non-linear relationships. By analyzing correlations, VIF, outliers, residuals, MSE, and R^2, the problems can be identified. Feature scaling, removing outliers, selecting important features, and adding relevant features can improve the linear regression model and reduce prediction error.

Q2. Email Spam Classification Using a Linear Model

(i) Reasons for Misclassification

Spam emails may be classified as non-spam because of:

1. Improper decision boundary – The linear model may have a boundary that is not suitable for separating spam and non-spam emails.
2. Imbalanced data – If there are many more non-spam emails than spam emails, the model may favor the non-spam class.
3. Poor features – Important spam indicators such as suspicious words, links, or excessive punctuation may not be included.
4. Wrong threshold – The default classification threshold of 0.5 may be too high for detecting spam.
5. Insufficient training data – The model may not have seen enough examples of different types of spam.

(ii) Examine Decision Boundary and Errors

First, check the confusion matrix:

```
from sklearn.metrics import confusion_matrix
```

y_pred = model.predict(X_test)

print(confusion_matrix(y_test, y_pred))

The confusion matrix helps identify False Negatives, where spam emails are incorrectly classified as non-spam.

Also check the model's probabilities:

prob = model.predict_proba(X_test)[:, 1]

If many spam emails have probabilities close to the decision threshold, the boundary needs adjustment.

(iii) Optimization Techniques

1. Adjust the Classification Threshold

For spam detection, we can use a lower threshold:

threshold = 0.4

y_new = (prob >= threshold).astype(int)

This can increase spam detection.

2. Feature Engineering

Add useful spam-related features such as:

- Number of suspicious words
- Number of links
- Number of special characters
- Email length
- Presence of words like "free", "offer", or "winner"

3. Balance the Training Data

Use class weights:

from sklearn.linear_model import LogisticRegression

model = LogisticRegression(class_weight='balanced')

model.fit(X_train, y_train)

This gives more importance to the minority spam class.

4. Re-train the Model

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

Then evaluate again using accuracy, precision, recall, and the confusion matrix.

Conclusion

The main problem is False Negative spam predictions. The model can be improved by adjusting the decision threshold, adding useful spam-related features, and training with balanced data. These techniques help the model detect more spam emails and reduce misclassification.

Q3. Naïve Bayes Classifier with Missing or Zero Probabilities

(i) Identify the Issue

Naïve Bayes calculates the probability of a class based on the probabilities of the features.

The main problems are:

1. Missing values – Some features may contain missing data.
2. Zero probability – If a feature value never appears in a particular class during training, its probability becomes 0.
3. Zero-frequency problem – Multiplying probabilities that include zero results in an overall probability of zero.

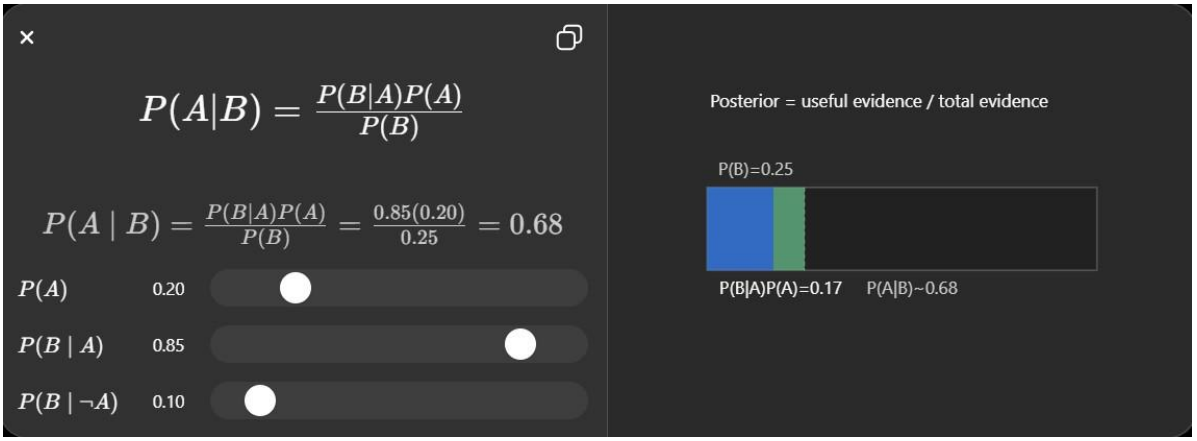
(ii) Effect on Classification

Suppose an email contains a word that never appeared in the spam training examples.

Then:

P(word | spam) = 0

Since Naïve Bayes multiplies the feature probabilities, the entire probability for the spam class can become zero.



As a result, the classifier may incorrectly select another class, causing **poor accuracy and misclassification**.

(iii) Debugging and Optimization

1. Handle Missing Values

```
df = df.dropna()
```

Or replace missing numerical values:

```
df = df.fillna(df.mean(numeric_only=True))
```

2. Apply Laplace Smoothing

Laplace smoothing prevents zero probabilities.

```
from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB(alpha=1.0)

model.fit(X_train, y_train)
```

Here, alpha=1 applies **Laplace smoothing**.

3. Check Probabilities

```
print(model.class_log_prior_)

print(model.feature_log_prob_)
```

This helps identify unusual probability values.

4. Evaluate the Improved Model

```
from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

Conclusion

The main issue is the **zero-frequency problem**, where an unseen feature produces a zero probability and can make the complete class probability zero. **Handling missing values and applying Laplace smoothing** prevents this problem and generally improves the Naïve Bayes classifier's accuracy.

Q4. Discriminant Function for Class Separation

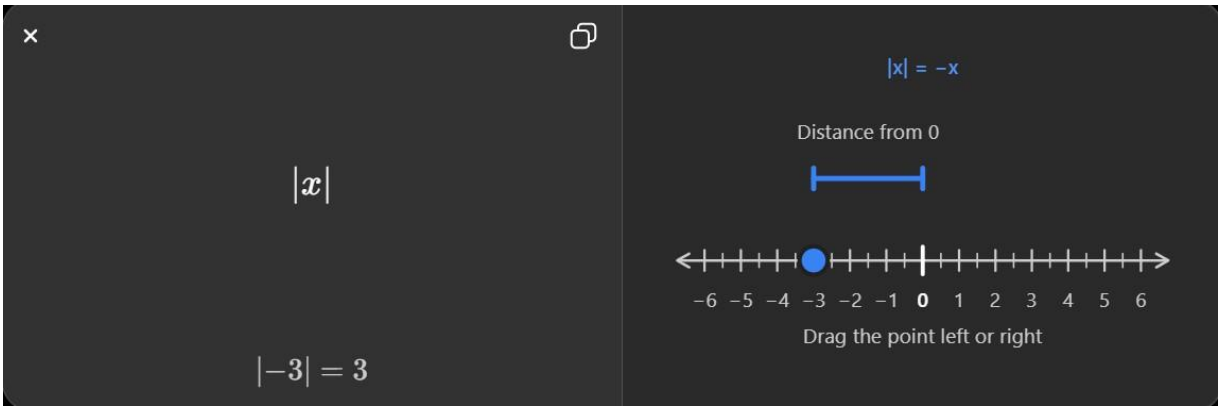
(i) Reasons for Poor Class Separation

The discriminant function may fail to separate two classes because of:

1. **Overlapping features** – Both classes have similar feature values.
2. **Incorrect weights** – The weights assigned to important features may be too low or incorrect.
3. **Different feature scales** – Features may have very different ranges.
4. **Irrelevant features** – Unnecessary features can affect the decision boundary.
5. **Insufficient training data** – The model may not learn the correct separation between classes.

(ii) Analyze Decision Boundary and Feature Contribution

A discriminant function can be represented as:



For a simple linear discriminant, the decision depends on the weighted combination of features.

- Check which samples are incorrectly classified.
- Plot the two classes to observe whether their features overlap.
- Examine the weights of each feature.
- A feature with a larger absolute weight has a greater influence on the decision.
- If the boundary passes through many samples from both classes, the separation is poor.

(iii) Improve the Discriminant Function

1. Adjust Weights

Optimize the feature weights using training data so that important features have suitable weights.

from sklearn.linear_model import LogisticRegression

```
model = LogisticRegression()
model.fit(X_train, y_train)
print("Feature weights:", model.coef_)
```

2. Normalize Features

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

This puts features on a similar scale.

3. Feature Transformation

Transform or create new features that provide better separation between the classes.

Examples:

- Polynomial features
- Log transformation
- Feature combinations

4. Remove Irrelevant Features

Remove features that contribute little to classification and retain the most useful features.

5. Re-train and Evaluate

```
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)
```

Conclusion

Poor class separation can result from **overlapping features, incorrect weights, different feature scales, and irrelevant features**. By analyzing the decision boundary and feature weights, then applying **weight adjustment, normalization, feature transformation, and feature selection**, the discriminant function can achieve better class separation and classification accuracy.

Q5. Bayesian Logistic Regression for Customer Churn Prediction

(i) Signs and Causes of Overfitting

Overfitting occurs when the model learns the training data too closely and performs poorly on new data.

Signs:

1. Very high training accuracy.
2. Much lower testing accuracy.
3. Low training error but high testing error.
4. Model gives unstable predictions for new customers.

Causes:

- Too many features.
- Small training dataset.
- Irrelevant or noisy features.
- Model is too complex.
- Insufficient regularization.

(ii) Analyze Training vs Testing Performance

Train and test accuracy can be compared as follows:

```
train_accuracy = model.score(X_train, y_train)

test_accuracy = model.score(X_test, y_test)

print("Training Accuracy:", train_accuracy)

print("Testing Accuracy:", test_accuracy)
```

For example:

Performance	Accuracy
Training	95%
Testing	78%

The large difference indicates **overfitting**.

We can also calculate cross-validation accuracy:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5)
```

```
print("CV Accuracy:", scores.mean())
```

(iii) Debugging and Regularization

1. L1 Regularization

L1 regularization can remove less important features by reducing their coefficients toward zero.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(
    penalty='l1',
    solver='liblinear',
    C=0.1
)
model.fit(X_train, y_train)
```

2. L2 Regularization

L2 regularization reduces large model coefficients and helps prevent overfitting.

```
model = LogisticRegression(
    penalty='l2',
    C=0.1
)
model.fit(X_train, y_train)
```

A smaller C means **stronger regularization**.

3. Cross-Validation

Use cross-validation to select the best model parameters:

```
from sklearn.model_selection import GridSearchCV
params = {'C': [0.01, 0.1, 1, 10]}
grid = GridSearchCV(
    LogisticRegression(),
    params,
    cv=5
)
```

```
grid.fit(X_train, y_train)

print("Best C:", grid.best_params_)
```

Conclusion

The model is overfitting when **training performance is much better than testing performance**. Overfitting can be reduced by using **L1/L2 regularization, cross-validation, feature selection, and suitable hyperparameter tuning**. These methods improve the model's ability to generalize to new customers.

Q6. Linear Regression with Different Feature Scales

(i) Effect of Feature Scaling Issues

In the sales prediction model, features have different scales:

- Advertising budget → measured in **thousands**
- Number of outlets → measured in **units**

When features have very different ranges, the regression model may give **unequal importance to features** and optimization can become less efficient.

Feature scaling puts all features into a comparable range.

(ii) Impact on Regression Coefficients

Without scaling, the coefficients may have very different numerical values because the input features use different units.

For example:

- Advertising budget = 100, 200, 300
- Number of outlets = 5, 10, 15

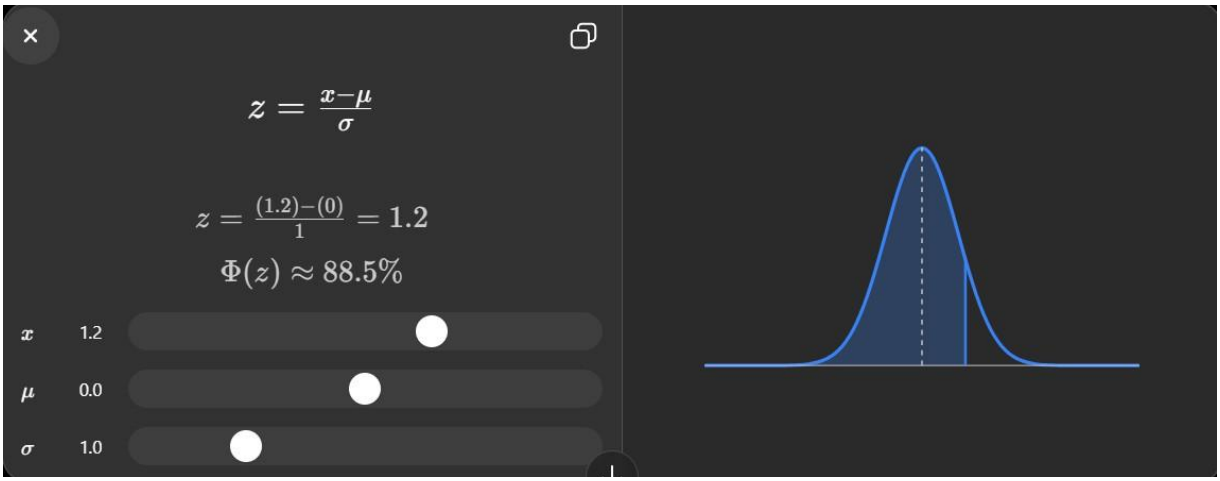
The coefficient values cannot be directly compared based only on their size.

Scaling makes the coefficients easier to interpret and can improve numerical stability.

(iii) Optimization Techniques

1. Standardization

Standardization converts features to approximately mean 0 and standard deviation 1.




```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

2. Normalization

Min-Max normalization converts values into a range, usually 0 to 1.

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3. Train the Regression Model

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
model = LinearRegression()
model.fit(X_train_scaled, y_train)
pred = model.predict(X_test_scaled)
print("MSE:", mean_squared_error(y_test, pred))
print("R2 Score:", r2_score(y_test, pred))
```

Conclusion

Different feature scales can affect the stability and efficiency of the regression model and make coefficient magnitudes difficult to compare. **Standardization or normalization** brings the features to comparable scales and can improve model performance and optimization.

Q7. Binary Classification Model for Fraud Detection

(i) Problem Caused by Class Imbalance

In fraud detection, the number of **non-fraudulent transactions is usually much higher** than fraudulent transactions.

This creates **class imbalance**.

Problems caused by imbalance:

1. The model becomes biased toward the majority class.
2. Fraudulent transactions may be classified as non-fraudulent.
3. Accuracy may appear high even when fraud detection is poor.
4. **False Negatives** can increase.

(ii) Evaluate Model Performance

Accuracy alone is not suitable for an imbalanced dataset. Use **precision, recall, and F1-score**.

```
from sklearn.metrics import precision_score, recall_score, f1_score

print("Precision:", precision_score(y_test, y_pred))

print("Recall:", recall_score(y_test, y_pred))

print("F1 Score:", f1_score(y_test, y_pred))
```

Precision: Measures how many transactions predicted as fraud are actually fraud.

Recall: Measures how many actual fraudulent transactions are correctly detected.

F1-score: Combines precision and recall into a single measure.

For fraud detection, **high recall is especially important** because missing a fraudulent transaction is costly.

(iii) Optimization Techniques

1. Oversampling

Increase the number of minority-class samples.

```
from imblearn.over_sampling import RandomOverSampler

ros = RandomOverSampler(random_state=42)

X_train_new, y_train_new = ros.fit_resample(X_train, y_train)
```

This gives the model more fraudulent examples during training.

2. Undersampling

Reduce the number of majority-class samples.

```
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(random_state=42)

X_train_new, y_train_new = rus.fit_resample(X_train, y_train)
```

3. Class Weighting

Give greater importance to the minority class.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(class_weight='balanced')

model.fit(X_train, y_train)
```

Conclusion

Class imbalance causes the fraud detection model to favor the majority **non-fraud** class. The model should be evaluated using **precision, recall, and F1-score**, rather than accuracy alone. **Oversampling, undersampling, and class weighting** can reduce bias and improve the detection of fraudulent transactions.

Q8. Naïve Bayes with Highly Correlated Features

(i) Effect of Correlated Features

Naïve Bayes assumes that the input features are **independent of each other** given the class.

When features are highly correlated:

1. The independence assumption is violated.
2. The same information may be counted multiple times.

3. Class probabilities can become inaccurate.
4. This may lead to incorrect classifications and reduced accuracy.

Example: House Size and Number of Rooms may contain similar information.

(ii) Analyze Feature Relationships

A correlation matrix can be used to identify highly correlated features.

```
import matplotlib.pyplot as plt
import seaborn as sns
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True)
plt.show()
```

If two features have a correlation close to **+1 or -1**, they are strongly related.

For example:

Feature A ↔ Feature B = 0.92

This indicates high correlation, so one of the features may be removed.

(iii) Optimization Techniques

1. Feature Selection

Remove one of the highly correlated features.

```
X = df.drop(columns=['Feature_B'])
y = df['Target']
```

Then train Naïve Bayes again:

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

2. Dimensionality Reduction Using PCA

PCA can transform correlated features into a smaller number of independent components.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
```

```
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
model = GaussianNB()
model.fit(X_train_pca, y_train)
print("Accuracy:", model.score(X_test_pca, y_test))
```

3. Compare Before and After

Compare the accuracy of:

- Original Naïve Bayes model
- Model after feature selection
- Model after PCA

Choose the approach that provides better test accuracy.

Conclusion

Highly correlated features violate the **independence assumption of Naïve Bayes** and may cause the same information to be counted more than once. A correlation matrix can identify these relationships. **Feature selection and PCA-based dimensionality reduction** can reduce the effect of correlated features and improve classification accuracy.

Q9. Logistic Regression with Overlapping Classes

(i) Cause of Overlapping Decision Boundaries

Logistic regression may fail to clearly separate two classes because:

1. **Features have overlapping distributions** – Both classes have similar feature values.
2. **Linear decision boundary is insufficient** – Logistic regression normally creates a linear boundary.
3. **Irrelevant features** may make separation difficult.
4. **Insufficient or noisy data** can cause classification errors.

(ii) Analyze Feature Space and Classification Errors

A scatter plot can be used to observe how the classes are distributed.

```
import matplotlib.pyplot as plt
```

```
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

If points from both classes are mixed together, the features do not provide clear separation.

Check classification errors using a confusion matrix:

```
from sklearn.metrics import confusion_matrix
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
```

The confusion matrix shows the number of correctly and incorrectly classified samples.

(iii) Improve the Model

1. Feature Transformation

Transform existing features to make the classes easier to separate.

Examples:

- Log transformation
- Square-root transformation
- Standardization

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. Polynomial Features

Polynomial features allow logistic regression to model **non-linear boundaries**.

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
```

Then train logistic regression:

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
model.fit(X_train_poly, y_train)
print("Accuracy:", model.score(X_test_poly, y_test))
```

3. Regularization

Regularization prevents the model from becoming unnecessarily complex.

```
model = LogisticRegression(
    penalty='l2',
    C=0.1
)
model.fit(X_train_poly, y_train)
```

A smaller C applies stronger regularization.

Conclusion

The main problem is **overlapping feature distributions**, which make it difficult for a simple linear boundary to separate the classes. By analyzing the feature space and errors, then applying **feature transformation, polynomial features, and regularization**, the logistic regression model can achieve better classification performance.

Q10. Discriminant Analysis with Underfitting

(i) Signs of Underfitting

Underfitting occurs when the model is too simple to learn the important patterns in the dataset.

Signs include:

1. **Low training accuracy**.
2. **Low testing accuracy**.
3. High training and testing errors.
4. The model fails to capture complex relationships between features and classes.
5. The decision boundary is too simple.

For example:

Performance	Accuracy
Training	65%
Testing	63%

When both training and testing accuracy are low, it indicates possible **underfitting**.

(ii) Analyze Model Bias and Feature Limitations

A simple discriminant model may have **high bias** because it assumes a limited relationship between features and classes.

Possible limitations:

- Important features are missing.
- The model cannot represent nonlinear relationships.
- The available features may not contain enough information.
- The decision boundary may be too simple.

We can examine feature importance or coefficients:

```
print(model.coef_)
```

Also compare training and testing performance:

```
print("Training Accuracy:", model.score(X_train, y_train))
print("Testing Accuracy:", model.score(X_test, y_test))
```

(iii) Optimization Strategies

1. Add Relevant Features

Add useful features that can better distinguish the classes.

```
X = df[['Feature1', 'Feature2', 'Feature3', 'Feature4']]
y = df['Target']
```

2. Increase Model Complexity

Use a more flexible discriminant model such as **Quadratic Discriminant Analysis (QDA)** instead of Linear Discriminant Analysis (LDA).

```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
model = QuadraticDiscriminantAnalysis()
model.fit(X_train, y_train)
```

```
print("Accuracy:", model.score(X_test, y_test))
```

QDA can create nonlinear decision boundaries.

3. Nonlinear Feature Transformation

Create polynomial features to capture nonlinear relationships.

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)

X_train_new = poly.fit_transform(X_train)

X_test_new = poly.transform(X_test)
```

Conclusion

The model is underfitting when both training and testing performance are low. This usually occurs because the model is too simple or important features are missing. Adding relevant features, increasing model complexity, and applying nonlinear transformations can reduce bias and improve classification accuracy.